

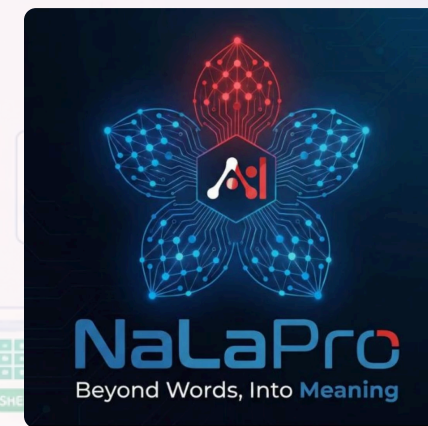
RAG Chatbot untuk Pelatihan Karyawan Baru

Final Project Bootcamp Indonesia AI

 Batch 10 Grup NaLaPro (NLP B)

Salam hangat untuk mentor dan peserta.

Kami dari tim NaLaPro mempersembahkan solusi praktis untuk mempercepat onboarding di industri FnB.



**Wahid Setio
Darmadi**



**M. Dhimas Agung
Sugiharto**



**Yasmin
Kamila**



**Eko Erwis
Wandoko**

RAG PIPELINE



DOCUMENTS



CHUNKING



VECTOR
DATABASE



RETRIEVAL



GENERATION



ANSWER

RETRIEVAL

LLM
GENERATE
ANSWER



Latar Belakang & Identifikasi Masalah

Kami mengembangkan project ini untuk menyelesaikan hambatan utama di proses onboarding karyawan baru pada Katering Yeyeti.

Onboarding lambat

Pelatihan memakan waktu panjang karena banyak materi dan simulasi praktis.

Beban baca dokumen

Karyawan harus membaca SOP, manual, dan daftar menu secara manual—membosankan dan tidak efisien.

Pertanyaan berulang

HR dan supervisor sering menjawab pertanyaan yang sama, menyita waktu operasional.

Pendekatan Solusi

Kami membangun chatbot berbasis Retrieval-Augmented Generation (RAG) agar karyawan bisa bertanya secara natural dan menerima jawaban yang selaras dengan dokumen resmi.



Interaksi natural

Karyawan gunakan bahasa sehari-hari — chatbot mengembalikan jawaban berbasis dokumen.



Retrieval yang andal

Dokumen diindeks ke vector DB untuk pencarian konteks yang relevan.



Jawaban sesuai SOP

Respons distandarisasi sehingga meminimalisir miskomunikasi.



Teknologi kunci: Llama-3.1-8b-instant (Groq) - Qdrant - Sentence Transformers.



Environment & Pipeline Sistem



RAG ONBOARDING CHATBOT

Asisten Informasi Karyawan – F&B Industry

ENHANCED RAG

F&B EMPLOYEE ONBOARDING

KATERING YEYETI CASE STUDY

Chatbot berbasis RAG untuk membantu karyawan baru memahami dokumen internal perusahaan secara interaktif. Jawaban akurat, cepat, dan berdasarkan dokumen resmi perusahaan.



TECH STACK



LangChain
Orchestration



SentenceTransformers
Embedding Model
MiniLM-L12-v2



Qdrant Cloud
Vector Database



Groq LLaMA 3.1
LLM
LLaMA 3.1 8B Instant



Streamlit
UI



Google Colab
T4 GPU



Google Drive
Storage



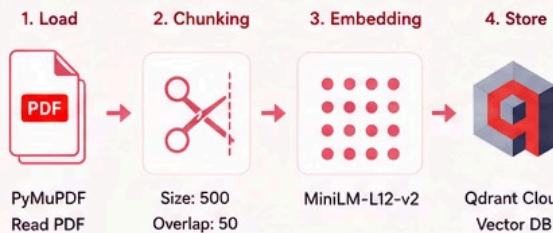
Google Sheets API
Logging



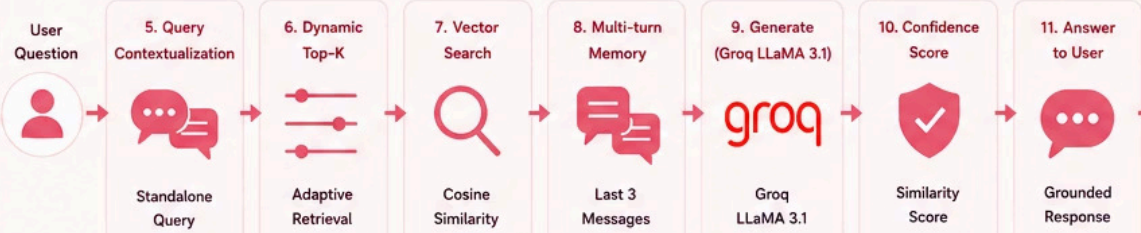
ROUGE Score
Evaluation

ENHANCED RAG PIPELINE

DATA INGESTION (OFFLINE)



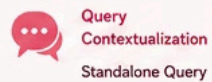
RETRIEVAL & GENERATION (ONLINE)



OUTPUT

- Grounded Answers
- Confidence Score
- Auto Logging

ENHANCED RAG FEATURES



Query Contextualization
Standalone Query



Dynamic Top-K
Adaptive Retrieval



Multi-turn Memory
Conversation Context
(Last 3 Messages)



Confidence Score
Similarity-Based Score



Anti-Hallucination Guardrail
Response Restricted to Source Documents



ROUGE Evaluation
Measure Answer Quality



LOG & MONITORING

Google Sheets API
Timestamp, Question, Answer, Company, Response Time



DATASET (KATERING YEYETI)

11 PDFs

31 Pages

145 Chunks

ESTIMASI PEMAKAIAN

~ 1.400 tokens / request
(960 input + 396 output)
LLaMA 3.1 8B Instant



EVALUATION

ROUGE Score
Untuk mengukur kualitas jawaban terhadap dokumen referensi.



BUILT WITH

Python • LangChain • Groq
Qdrant • Google Colab • Streamlit
Google Drive • Google Sheets
SentenceTransformers

★ Enhanced RAG = Akurasi Lebih Tinggi • Hallucination Terkendali • Percakapan Lebih Natural • Siap untuk Skalabilitas

Pipeline Aplikasi



Diagram menunjukkan tiga langkah inti RAG: Ingest dokumen → Retrieval dari vector DB → Generation oleh LLM yang dikondisikan pada konteks terambil.

Ingest Document

1

Data Loading (Pengumpulan Data)

2

Chunking (Pemotongan Data)

3

Embedding (Transformasi ke Vektor)

4

Indexing (Penyimpanan di Vector Database
- Qdrant)

Retrieve Chunks

Pertanyaan Pengguna Menjadi Vektor

Ketika pengguna memasukkan pertanyaan, sistem tidak mencari berdasarkan kata kunci (*keyword matching*), melainkan mengubah pertanyaan tersebut menjadi bentuk yang sama dengan data di database: **Vektor** (angka).

Pemilihan Top-K

Setelah semua *chunk* dihitung kemiripannya, sistem akan mengambil nilai : key jumlah *chunk* dengan skor tertinggi.

1

2

3

Semantic Search (Pencarian Berdasarkan Makna)

Sistem membandingkan vektor pertanyaan dengan ribuan (atau jutaan) vektor *chunk* yang sudah disimpan di database.

Retrieve Chunks – Text Splitter

	Experiment 01	Experiment 02	Experiment 03	Experiment 04
function	RecursiveCharacterTextSplitter()	RecursiveCharacterTextSplitter()	RecursiveCharacterTextSplitter()	RecursiveCharacterTextSplitter()
chunk_size	500	300	400	500
chunk_overlap	50	50	50	80
Total chunks	141	251	181	151

- **Chunk_Size** berfungsi untuk menentukan ukuran potongan text semakin kecil chunk yang dihasilkan akan semakin banyak.
- **Chunk_overlap** berfungsi untuk menentukan seberapa banyak text akhir potongan pertama yang diulangi di awal potongan kedua. semakin besar nilai chunk_overlap maka total chunk akan semakin besar
- Untuk menghemat token API kami menggunakan **Experiment 01** dan ketika kami melakukan test hasil yang didapat sudah sesuai dengan dokumen

Pemilihan Top-K

Top-K adalah parameter yang menentukan berapa banyak jumlah dokumen atau potongan teks (chunks) paling relevan yang akan diambil oleh sistem dari database vektor (Qdrant).

- **Jika Semakin Besar**

1. Token API yang digunakan akan semakin banyak
2. Informasi yang akan diterima oleh LLM akan semakin banyak

- **Jika Semakin Kecil**

1. Token API yang digunakan akan semakin hemat
2. Informasi yang diterima oleh LLM akan semakin sedikit
3. Bisa memicu tidak terbaca informasi yang ada

1

Top-K = 3

Contoh

pertanyaan : " Apa ada Menu Nasi Cumi Hitam ? "

Jawaban : dari 5x test, hasil jawaban bervariasi yaitu 3x jawaban betul dan 2x jawaban bahwa menu tersebut tidak ada di dalam SOP

2

Top-K = 11

Contoh

pertanyaan : " Apa ada Menu Nasi Cumi Hitam ? "

Jawaban : dari 5x test , Model selalu menjawab sesuai dengan Dokumen yang ada.

3

Penentuan menggunakan Top-k = 11

Efek samping yang disepakati :

1. Penggunaan token lebih boros
2. Disetujui boros karena ketepatan diprioritaskan daripada penghematan Token API

Generate Answer

1

Prompt Construction (Penyusunan Konteks)

Sistem menggabungkan beberapa elemen ke dalam satu *prompt* untuk dikirim ke LLM

2

LLM Inference (Pemrosesan)

LLM membaca seluruh isi *prompt* tersebut. Ia tidak lagi mencari di database, melainkan menggunakan kemampuan bahasanya untuk memahami hubungan antara konteks yang diberikan dengan pertanyaan pengguna.

3

Synthesis (Sintesis Jawaban)

AI merumuskan jawaban yang koheren, natural, dan (idealnya) faktual. AI akan mencoba menjawab dengan gaya bahasa yang diinginkan sambil tetap berpegang pada fakta yang ada di dalam *chunks*.

DOCUMENTS

CHUNKING

EMBEDDING

VECTOR
DATABASE

RETRIEVAL

LLM
GENERATION

ANSWER

Proses Pencarian Jawaban

Pada proses pencarian jawaban di Qdrant kami, menentukan menggunakan function **query_points**. function yang kami coba adalah **similarity_search_with_score** dan **query_points** ,penentuan menggunakan **query_points** dikarenakan faktor kecepatan dan performa pada proses pencarian ini, karena aplikasi yang kami buat merupakan chatbot maka selain ketepatan jawaban faktor kecepatan juga kami pertimbangkan dan **query_points** rata-rata lebih cepat 30%-50% daripada **similarity_search_with_score** .

similarity_search_with_score

- Hasil Jawaban

Tepat sesuai dengan Dokumen

- Kecepatan Jawaban

Agak Lambat

- Contoh :

- pertanyaan 1 : sebutkan keseluruhan menunya (Response time = 1.8 s)
- pertanyaan 2 : pagi kak saya baru aja keterima kerja di katering ini sebagai staf dapur (Response time = 2.1 s)
- pertanyaan 3 : aku masuk jam berapa ya kak?(Response time = 1.29 s)
- pertanyaan 4 : kalau aku ga masuk itu gmn kak? misalnya sakit atau izin? terus ambil cuti bisa?(Response time = 1.74 s)
- pertanyaan 5 : untuk seragam saya gimana kak? (Response time = 1.68 s)

query_points (Used)

- Hasil Jawaban

Tepat sesuai dengan Dokumen

- Kecepatan Jawaban

Cepat

- Contoh :

- pertanyaan 1 : sebutkan keseluruhan menunya (Response time = 2.53 s)
- pertanyaan 2 : pagi kak saya baru aja keterima kerja di katering ini sebagai staf dapur (Response time = 2.09 s)
- pertanyaan 3 : aku masuk jam berapa ya kak?(Response time = 2.05 s)
- pertanyaan 4 : kalau aku ga masuk itu gmn kak? misalnya sakit atau izin? terus ambil cuti bisa?(Response time = 2.82 s)
- pertanyaan 5 : untuk seragam saya gimana kak? (Response time = 2.08 s)

	Groq	Gemini
Model	llama-3.1-8b-instant	gemini-2.5-flash-lite
Token	131,072	8,000 (Free Rate)
Kendala	Untuk develop lebih mudah dan murah	Selalu kena limit ketika development
Pilihan	True	



MENGAPA MEMILIH GROQ?

Perbandingan Limit & Pengalaman Pengembangan

Selama fase eksperimen dan pengujian RAG Chatbot Onboarding Karyawan



LLaMA 3.1 8B

VS



1 FASE EKSPERIMEN

Tim mencoba beberapa LLM untuk kebutuhan RAG onboarding karyawan.



Gemini
(Flash / Pro)

→



LLaMA 3.1 8B

2 TANTANGAN PENGEMBANGAN



Banyak testing dan iterasi



Evaluasi 71+ pertanyaan



Ratusan request



Ribuan token context panjang



Pengujian intensif ini membuat beberapa layanan cepat mencapai batas penggunaan (rate limit / quota).

3 KEBUTUHAN KONTEKS SISTEM RAG



11 Dokumen



31 Halaman



145 Chunks



225 Request

Rata-rata Penggunaan Token (Groq)

Input Token	Output Token	Total Token
1.612 /token	145 /token	1.757 /token

Peak Usage: 5.261 token / request • Total Processed: 395.325 token

4 PERBANDINGAN PENGALAMAN PENGEMBANGAN

KRITERIA	GEMINI (FREE / TIER TERBATAS)	GROQ (LLaMA 3.1 8B)
 Limit / Quota Penggunaan	 Cepat mencapai limit Free tier terbatas per hari / per menit sehingga pengujian terhambat.	 Lebih longgar & stabil Mendukung pengujian berulang tanpa cepat mencapai batas.
 Kecepatan Respons	 Baik Tergantung beban & antrian server.	 Sangat cepat Inferensi ultra-fast dari infrastruktur Groq.
 Context Window (RAG)	 Baik Mendukung context panjang.	 Sangat baik Hingga 128K token, ideal untuk RAG.
 Stabilitas untuk Eksperimen	 Sering terbatas Banyak percobaan terhenti karena limit / quota harian.	 Sangat stabil Mendukung eksperimen intensif dari awal hingga demo.
 Biaya Operasional	 Lebih tinggi Biaya per 1M token relatif lebih mahal.	 Lebih hemat Biaya per 1M token lebih rendah.

KESIMPULAN



PILIHAN AKHIR: GROQ LLaMA 3.1

Groq dipilih karena mampu menangani konteks dokumen yang besar dengan limit penggunaan yang lebih longgar, kecepatan respons tinggi, dan biaya operasional lebih hemat.

5 HASIL PENGUJIAN DENGAN GROQ



225 Request

Percakapan berhasil dievaluasi.



3.05 detik

Rata-rata response time.



1.757 token

Rata-rata total token per request.



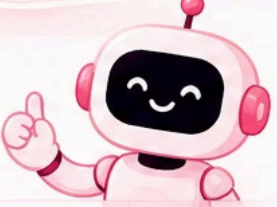
395K+ token

Total token diproses selama pengujian.

6 KESIMPULAN PEMILIHAN PLATFORM



Karena kebutuhan RAG yang memerlukan context panjang, banyak iterasi, dan evaluasi ratusan pertanyaan, Groq memberikan fleksibilitas pengujian yang jauh lebih baik dibandingkan Gemini yang lebih cepat mencapai limit. Hal ini memastikan proses pengembangan berjalan lebih lancar, efisien, dan berhasil mencapai demo final tanpa hambatan kuota.



★ Dengan limit yang lebih longgar, kecepatan tinggi, dan biaya yang lebih hemat, Groq adalah pilihan terbaik untuk implementasi Chatbot Onboarding Karyawan berbasis RAG di Katering Yeyeti.



Lebih Stabil • Lebih Cepat • Lebih Hemat

Katering Yeyeti

Tanyakan apa saja tentang perusahaan, benefit, prosedur, dan kebijakan kerja.

 Halo! Saya asisten onboarding Katering Yeyeti yang ramah dan amanah. 😊 Silakan tanyakan apa saja seputar perusahaan — mulai dari hak karyawan, jam kerja, SOP, kebijakan halal, hingga menu dan produk kami.

 HALLO OM

 Halo Om! Saya senang membantu. Apa yang ingin kamu tanyakan tentang Yeyeti Katering?

Ketik pertanyaan kamu di sini...



Hasil Implementasi & Live Demo

Kami menunjukkan aplikasi **Streamlit** — demo live menampilkan kemampuan multi-turn dan respons berbasis dokumen.

Silakan akses demo live!

01

Multi-turn memory

Chatbot mengingat 3 percakapan terakhir untuk konteks percakapan berkelanjutan.

02

Kecepatan & Confidence

Latency diukur per request; sistem menandai jawaban dengan confidence rendah untuk verifikasi manusia.

03

Fleksibilitas jawaban

Menjawab pertanyaan menu, harga, aturan halal, hingga SOP layanan pelanggan.





HASIL EVALUASI & DEMO CHATBOT ONBOARDING KARYAWAN

Evaluasi performa RAG Chatbot berbasis dokumen internal Katering Yeyeti



TOTAL CONVERSATIONS

118

Percakapan Tercatat



EVALUATED QUESTIONS

88

Pertanyaan Dievaluasi



AVERAGE RESPONSE TIME

3.05 s

Rata-rata Respons



BEST RELEVANCE SCORE

0.632

Skor Tertinggi



LOWEST RELEVANCE SCORE

0.195

Skor Terendah



AVERAGE RELEVANCE SCORE

0.457

Skor Rata-rata

⚡ RESPONSE TIME PERFORMANCE



TERCEPAT

0.90

Seconds

Pertanyaan:
"Iya menu
rekomendasi apa aja?"



RATA-RATA

3.05

Seconds

Normal operational
performance



TERLAMBAT

23.51

Seconds

Pertanyaan:
"Cuti sebulan
berapa kali?"

📊 RELEVANCE SCORE ANALYSIS



SKOR TERTINGGI

0.632

Pertanyaan:
"Saya bukan supervisor,
hanya tim dapur"



SKOR RATA-RATA

0.457

Kategori:

- Tinggi : 29 pertanyaan
- Sedang : 30 pertanyaan
- Rendah : 12 pertanyaan



SKOR TERENDAH

0.195

Pertanyaan:
"RPH itu apa?"

🔍 TEMUAN MENARIK

KEKUATAN CHATBOT

- ✓ Informasi menu katering
- ✓ SOP dasar perusahaan
- ✓ Pertanyaan onboarding karyawan
- ✓ Kebijakan operasional
- ✓ Informasi produk dan layanan

AREA YANG PERLU DITINGKATKAN

- ⚠ Pertanyaan di luar dokumen
- ⚠ Singkatan atau istilah khusus
- ⚠ Data yang tidak tersedia pada knowledge base
- ⚠ Pertanyaan ambigu tanpa konteks

🔄 KESESUAIAN INPUT vs OUTPUT



- ✓ Mayoritas jawaban tetap relevan terhadap pertanyaan pengguna.
- ✓ Chatbot mampu mengakui keterbatasan jika dokumen tidak tersedia.
- ✓ Tidak ditemukan halusinasi besar yang menghasilkan informasi berbahaya.
- ✓ Jawaban cenderung konservatif dan mengacu pada dokumen sumber.

💡 KESIMPULAN DEMO



RAG Chatbot berhasil menjawab pertanyaan onboarding karyawan berdasarkan dokumen internal perusahaan.

- ✓ Respons cepat dengan rata-rata 3.05 detik.
- ✓ Tingkat relevansi jawaban dalam kategori cukup baik hingga baik (0.457).
- ✓ Mengurangi beban tanya jawab berulang kepada HR dan supervisor.
- ✓ Arsitektur RAG menjaga jawaban tetap berbasis dokumen sehingga risiko halusinasi dapat diminimalkan.

DEMO RESULT: SUCCESSFUL IMPLEMENTATION



118 Conversations



3.05s Avg Response



0.457 Avg Relevance Score



Document-Grounded Answers

LOGs CHATBOT secara realtime silakan klik link ini: https://docs.google.com/spreadsheets/d/1g0xJGN687-eDW9XexmqvKW-mtjT7AWqsx_X65AMkpeU/edit?gid=0#gid=0



Manfaat & Dampak untuk Katering Yeyeti

1

Efisiensi waktu & biaya

Mempercepat onboarding sehingga karyawan cepat produktif dan mengurangi jam pelatihan langsung.

2

Skalabilitas

Metode bisa diterapkan di cabang lain dan industri berbeda; update dokumen cukup dilakukan di database.

3

Otomatisasi dukungan internal

HR & supervisor terbebas dari pertanyaan dasar berulang—fokus pada tugas strategis.

Langkah Selanjutnya & Roadmap

1

Short term

Optimasi retrieval, tuning threshold confidence, dan pengayaan dokumentasi internal, improving multi-turn memory

2

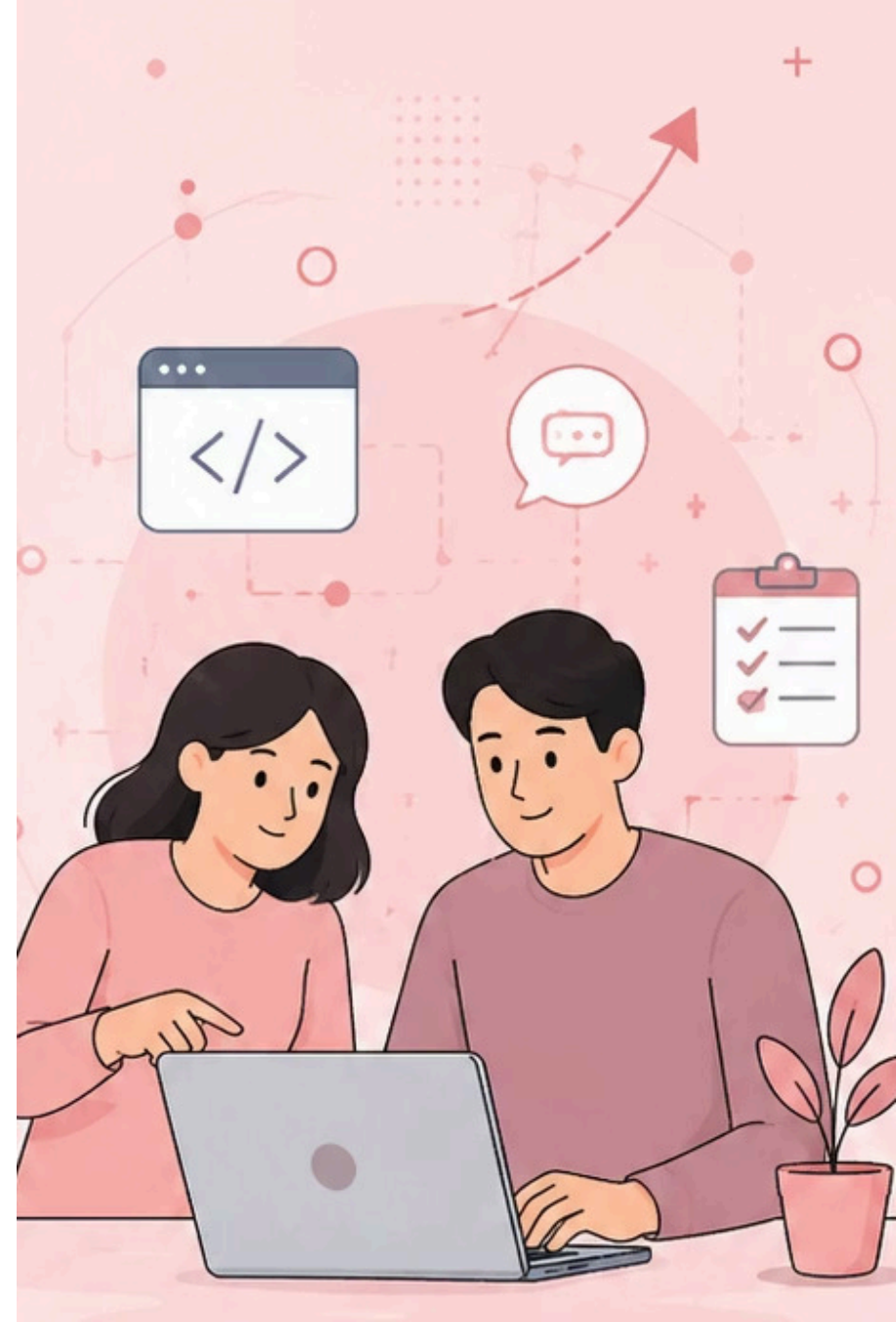
Mid term

Integrasi dengan HRIS, pelacakan KPI onboarding, dan dukungan multi-bahasa bila diperlukan.

3

Long term

Model on-premise untuk data sensitif, otomatis update dokumen via pipeline CI/CD, dan skala ke jaringan cabang.



Sesi Tanya Jawab & Penutup

Terima kasih atas waktu dan bimbingannya.

Kami membuka diskusi dan umpan balik.

https://github.com/nalapro/FinalProject_AI_Bootcamp (REPO)

<https://finalprojectaibootcamp-katering.streamlit.app> (DEMO)



Kontak

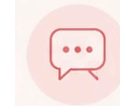
Tim NaLaPro — siap diskusi teknis dan penerapan skala.



Ucapan Penutup

Kami berharap solusi ini membantu mempercepat onboarding dan meningkatkan konsistensi operasional di Katering Yeyeti.

Q&A & PENUTUP



Terima kasih
(Indonesia)



Thank You
(English)



ありがとう
(日本語)



谢谢
(中国)



Спасибо
(Русский)



شكراً
(العربية)



धन्यवाद
(हिन्दी)



감사합니다
(한국어)



Matur nuwun
(Jawa)

